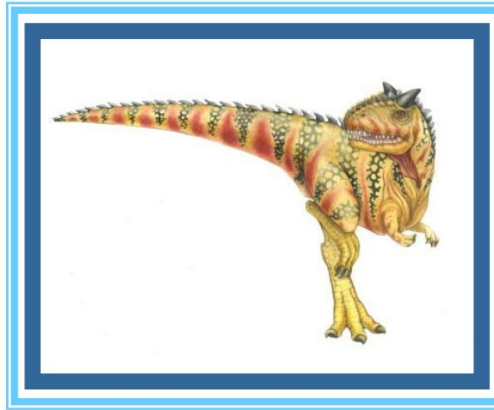


الفصل الرابع: الخيوط





الفصل الرابع: الخيوط

□ نظرة عامة

□ برمجة متعددة النواة □ نماذج متعددة

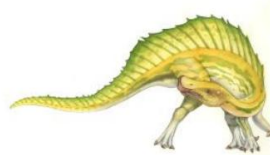
الخيوط

□ مكتبات الخيوط

□ الترابط الضمني

□ مشاكل الخيوط

□ أمثلة على أنظمة التشغيل





الفصل الرابع: الخيوط

Pos	Model	L2 Cache + L3 Cache	TDP (Watt)	MHz - Turbo	Cores / Threads	Perf. Rating
☐ 1	AMD Ryzen Threadripper PRO 3995WX	32MB + 256MB	280	2700 - 4200	64/128	82.2
☐ 2	AMD Ryzen Threadripper PRO 3975WX	16MB + 128MB	280	3500 - 4200	32/64	78.5
☐ 3	AMD Ryzen 9 5950X	8MB + 64MB	105	3400 - 4900	16/32	75.7
☐ 4	AMD Ryzen Threadripper 2970WX	12MB + 64MB	250	3000 - 4200	24/48	~54 ^{73%}
☐ 5	Intel Core i9-12900K	14MB + 20MB	125	3200 - 5200	8/24	78.7
☐ 6	AMD Ryzen 9 3950X	8MB + 64MB	105	3500 - 4700	16/32	61.4
☐ 7	AMD Ryzen 9 5900X	6MB + 64MB	105	3700 - 4800	12/24	67.7
☐ 8	Intel Core i9-10980XE	18MB + 24.8MB	165	3000 - 4600	18/36	63.5
☐ 9	Intel Core i9-9980XE	18MB + 24.8MB	165	3000 - 4400	18/36	~61.3 ^{91%}
☐ 10	AMD Ryzen Threadripper 2950X	8MB + 32MB	180	3500 - 4400	16/32	~51.7 ^{73%}
☐ 11	AMD Ryzen 9 3900X	6MB + 64MB	105	3800 - 4600	12/24	58.1
☐ 12	Intel Core i7-12700H	+ 24MB	45	2700 - 4600	14/20	~64.2 ^{36%}
☐ 13	Intel Core i5-12600K	9.5MB + 20MB	125	3700 - 4900	6/16	68.4
☐ 14	Intel Core i9-11900K	4MB + 16MB	125	3500 - 5300	8/16	63.7
☐ 15	Intel Core i9-11900KB	10MB + 24MB	65	3300 - 5300	8/16	
☐ 16	AMD Ryzen 7 5800X	4MB + 32MB	105	3800 - 4700	8/16	66.2
☐ 17	Intel Core i9-10900K	2.5MB + 20MB	125	3700 - 5300	10/20	59.4
☐ 18	AMD Ryzen 9 3900	6MB + 64MB	65	3100 - 4300	12/24	56
☐ 19	Intel Core i9-10850K	2.5MB + 20MB	125	3600 - 5200	10/20	59.1





أهداف

□ لتقديم مفهوم الخيط -وحدة أساسية لاستخدام وحدة المعالجة المركزية التي تشكل أساس أنظمة الكمبيوتر متعددة الخيوط

□ لمناقشة واجهات برمجة التطبيقات الخاصة بمكتبات الخيوط Java وWindows وPthreads

□ لاستكشاف العديد من الاستراتيجيات التي توفر الترابط الضمني

□ دراسة القضايا المتعلقة بالبرمجة متعددة الخيوط

□ لتغطية دعم نظام التشغيل للخيوط في Windows و

لينكس





تحفيز

□ معظم التطبيقات الحديثة متعددة الخيوط

□ يتم تشغيل الخيوط داخل التطبيق

□ يمكن تنفيذ مهام متعددة باستخدام التطبيق من خلال خيوط منفصلة

□ تحديث العرض

□ جلب البيانات

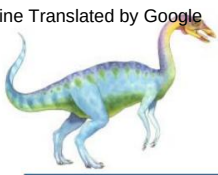
□ التدقيق الإملائي □ الرد على طلب الشبكة

□ عملية إنشاء الوزن الثقيل في حين أن إنشاء الخيوط خفيف الوزن

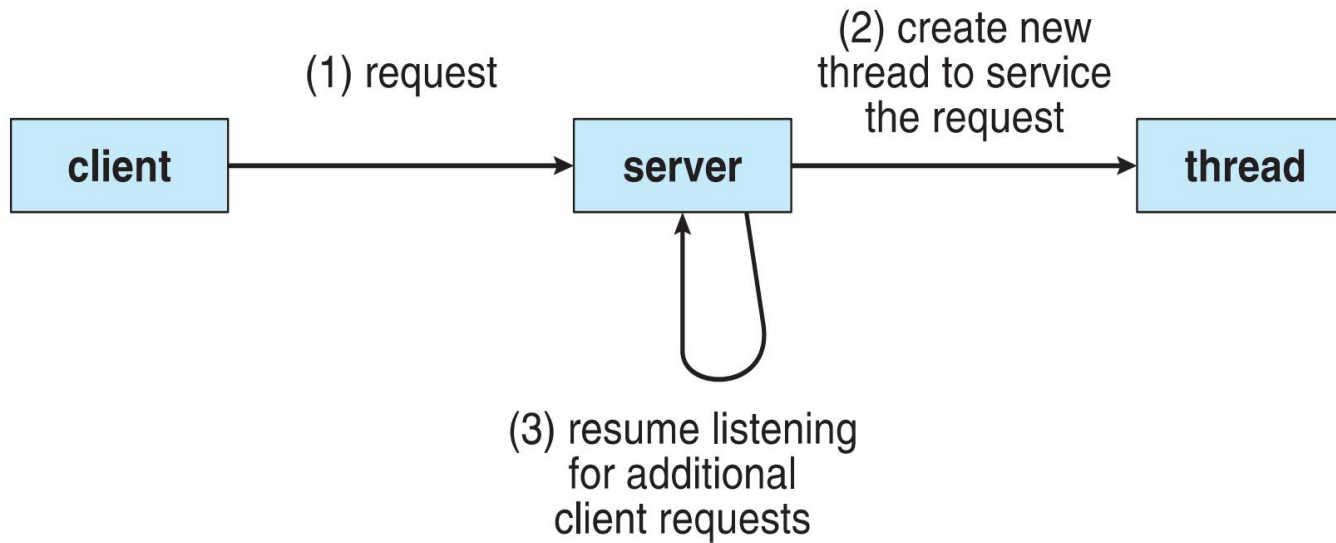
□ يمكن تبسيط الكود وزيادة الكفاءة

□ عادةً ما تكون النوى متعددة الخيوط





هندسة الخادم متعدد الخيوط





فوائد

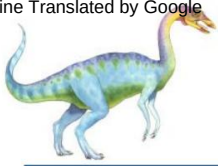
□ الاستجابة -قد تسمح باستمرار التنفيذ إذا تم حظر جزء من العملية، وهو أمر مهم بشكل خاص لمواجهة المستخدم

□ مشاركة الموارد -تتشارك الخيوط موارد العملية، وهو أمر أسهل من مشاركة الذاكرة أو تمرير الرسائل

□ الاقتصاد -أرخص من إنشاء العملية، والتبديل بين الخيوط أقل تكلفة من التبديل بين السياقات

□ إمكانية التوسع -يمكن للعملية الاستفادة من هياكل المعالجات المتعددة





برمجة متعددة النواة

□ أنظمة متعددة النواة أو متعددة المعالجات تضع ضغوطًا على
بالنسبة للمبرمجين، تشمل التحديات ما يلي:

□ تقسيم الأنشطة

□ التوازن

□ تقسيم البيانات

□ اعتماد البيانات

□ الاختبار والتصحيح

□ يعني أن النظام يمكنه تنفيذ أكثر من مهمة في وقت واحد

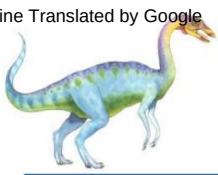
التوازي

□ يدعم أكثر من مهمة واحدة ويحقق التقدم

التزامن

□ معالج/نواة واحدة، جدول توفر التزامن





برمجة متعددة النواة (تابع)

أنواع التوازي

التوازي في البيانات - توزيع مجموعات فرعية من نفس البيانات عبر عدة أنوية، ونفس العملية على

كل منها

التوازي في المهام - توزيع الخيوط عبر النوى، كل منها

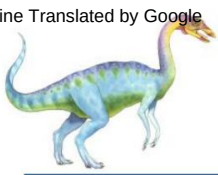
الخيوط يؤدي عملية فريدة

مع زيادة عدد الخيوط، يزداد أيضاً الدعم المعماري للخيوط

الخيوط للأجهزة - معالجة المركزية على أنوية بالإضافة إلى

فكر في Oracle SPARC T4 مع 8 أنوية و 8 خيوط أجهزة لكل نواة





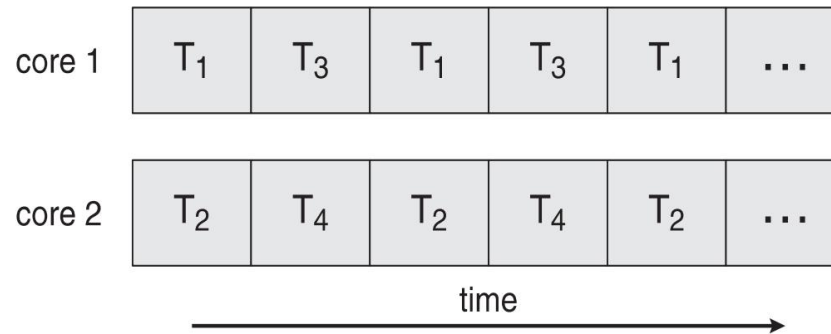
التزامن مقابل التوازي

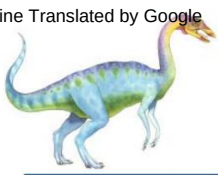


التنفيذ المتزامن على نظام أحادي النواة:

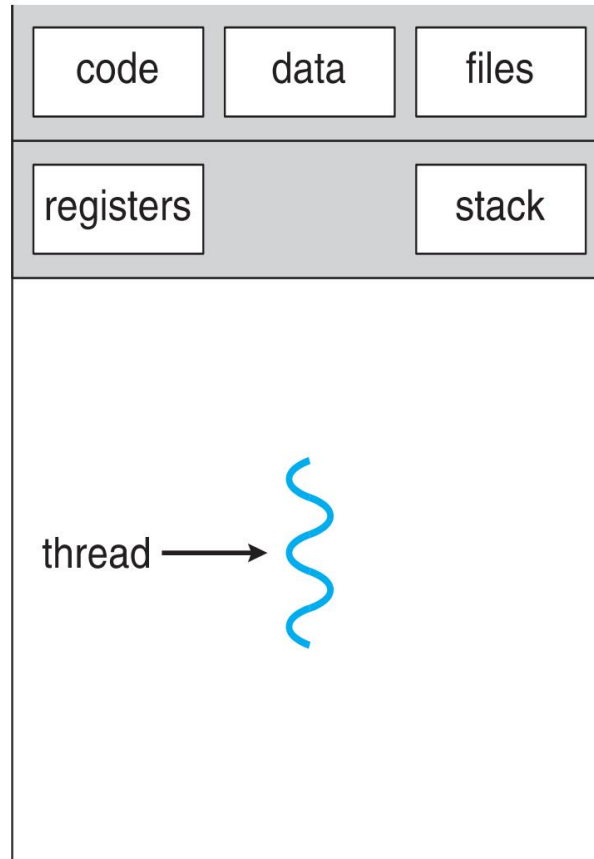


التوازي على نظام متعدد النواة:

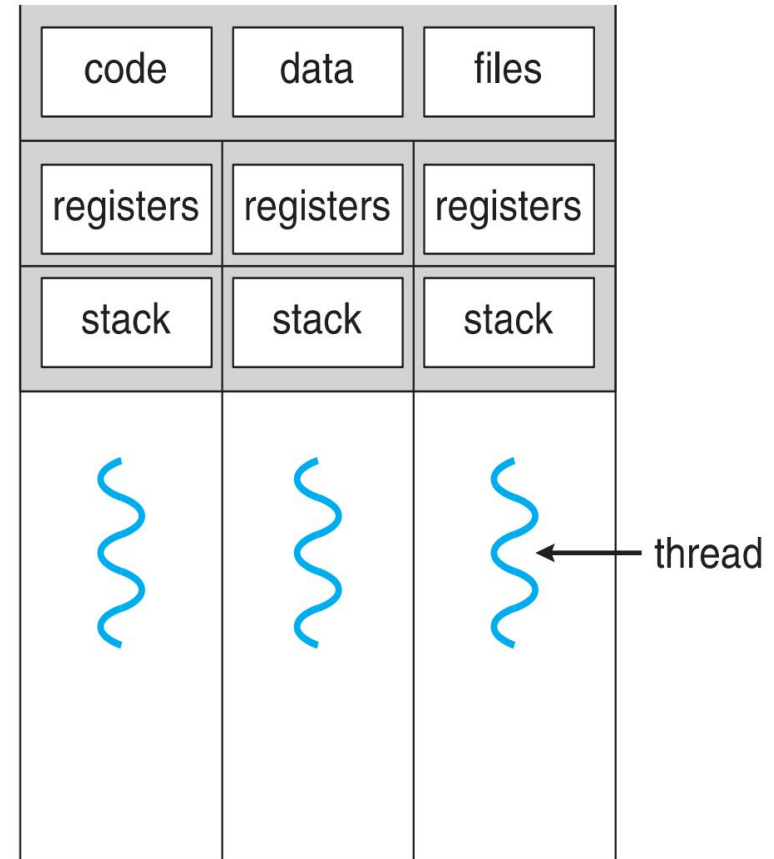




العمليات أحادية ومتعددة الخيوط

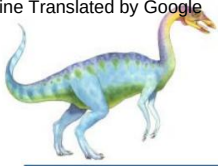


single-threaded process



multithreaded process





قانون أمدال

- يحدد مكاسب الأداء من إضافة أنوية إضافية إلى تطبيق يحتوي على مكونات تسلسلية ومتوازية
- يؤثر الجزء التسلسلي
- نوى المعالجة

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- وهذا يعني أنه إذا كان التطبيق 75% متوازيًا / 25% تسلسليًا، فإن الانتقال من 1 إلى 2 نواة يؤدي إلى تسريع يصل إلى 1.6 مرة
- يقترب من اللانهاية، ويقترح التسارع من 1 /

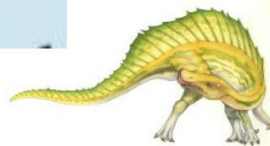
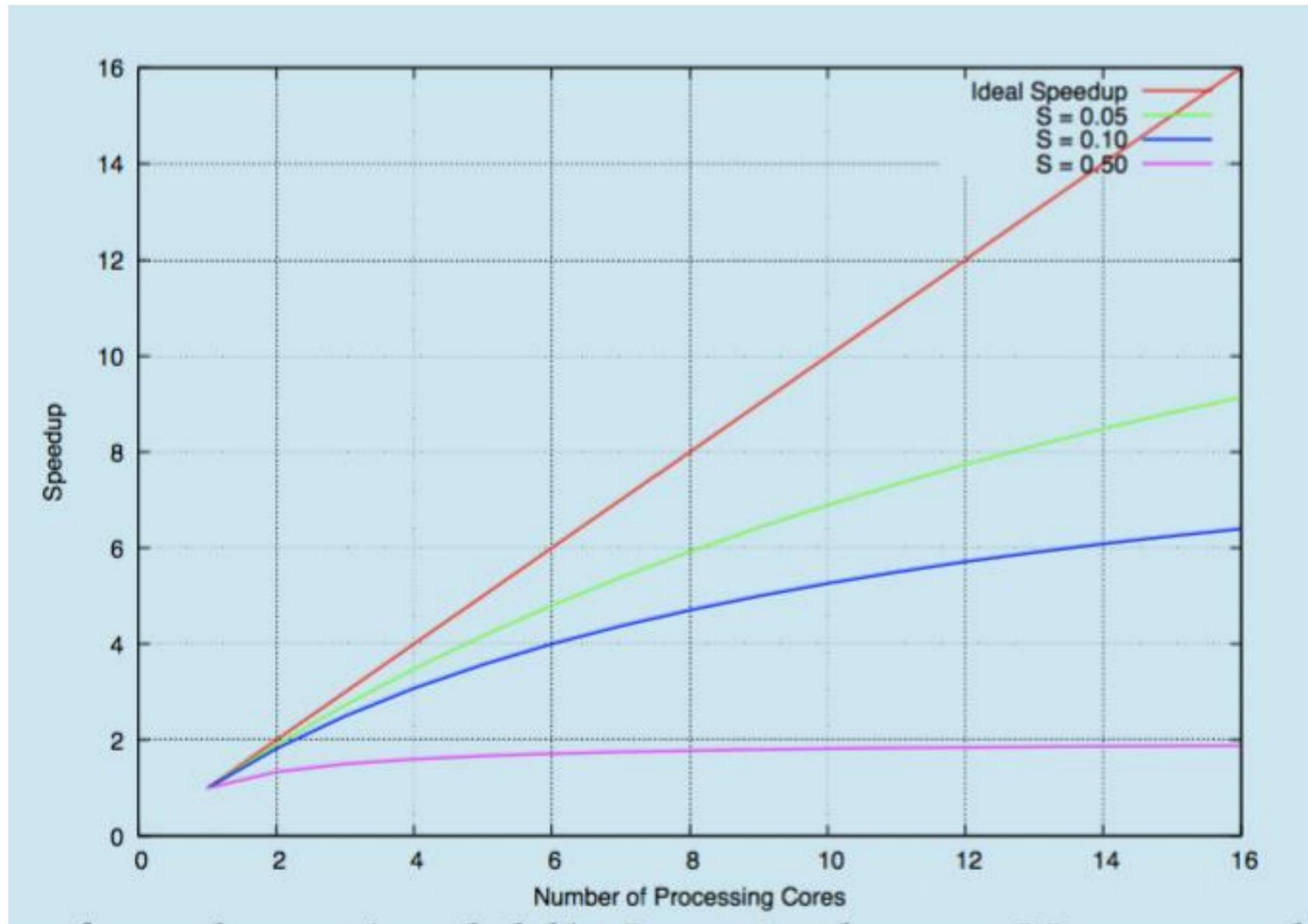
يؤثر الجزء التسلسلي من التطبيق بشكل غير متناسب على الأداء المكتسب من خلال إضافة أنوية إضافية

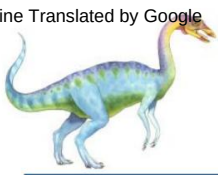
- ولكن هل يأخذ القانون في الاعتبار أنظمة متعددة النواة المعاصرة؟





قانون أمدال





خيوط المستخدم وخيوط النواة

❑ مؤشرات ترابط المستخدم - تتم الإدارة بواسطة مكتبة مؤشرات الترابط على مستوى المستخدم ❑ ثلاث

مكتبات مؤشرات ترابط أساسية:

❑ POSIX Pthreads

❑ خيوط الويندوز

❑ خيوط جافا

❑ خيوط النواة - مدعومة بواسطة النواة

❑ أمثلة - جميع أنظمة التشغيل للأغراض العامة تقريبًا، بما في ذلك:

❑ ويندوز

❑ سولاريس

❑ لينكس

❑ Tru64 يونكس

❑ نظام التشغيل ماك أو إس إكس





نماذج متعددة الخيوط

□ من العديد إلى واحد

□ واحد لواحد

□ من العديد إلى العديد





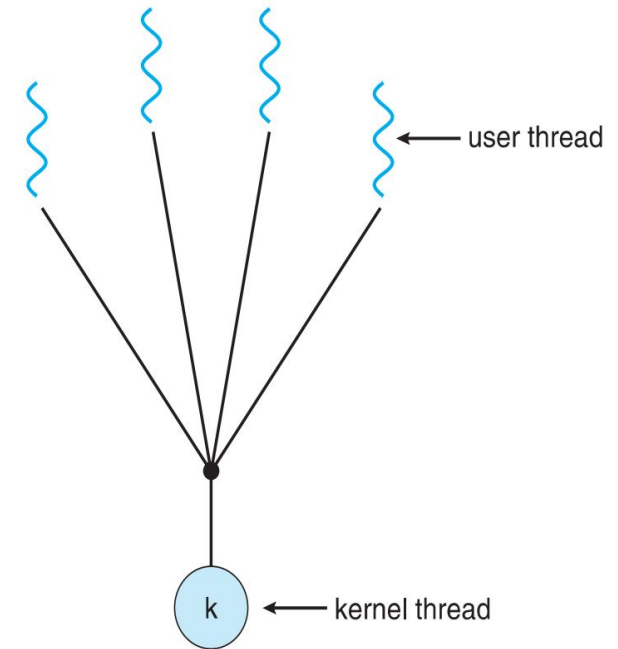
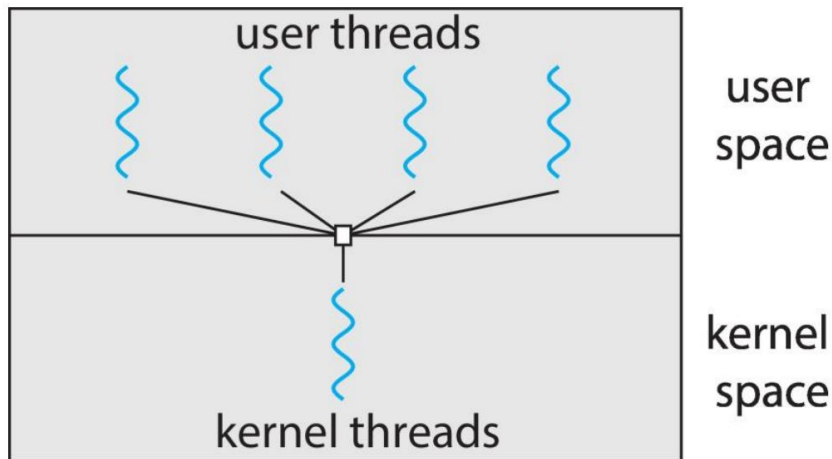
من العديد إلى واحد

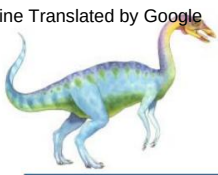
تم تعيين العديد من مؤشرات الترابط على مستوى المستخدم إلى مؤشر ترابط نواة واحد

- يؤدي حظر مؤشر ترابط واحد إلى حظر جميع المؤشرات
- قد لا تعمل عدة خيوط بالتوازي على نظام متعدد النواة لأنه قد يكون هناك خيوط واحدة فقط في النواة في كل مرة
- هناك عدد قليل من الأنظمة التي تستخدم هذا النموذج حاليًا
- أمثلة:

□ خيوط سولاريس الخضراء

□ خيوط GNU المحمولة





واحد لواحد

□ يتم ربط كل مؤشر ترابط على مستوى المستخدم بمؤشر ترابط النواة

□ إنشاء مؤشر ترابط على مستوى المستخدم ينشئ مؤشر ترابط أساسي □ مزيد من التزامن مقارنة بالتعدد إلى واحد

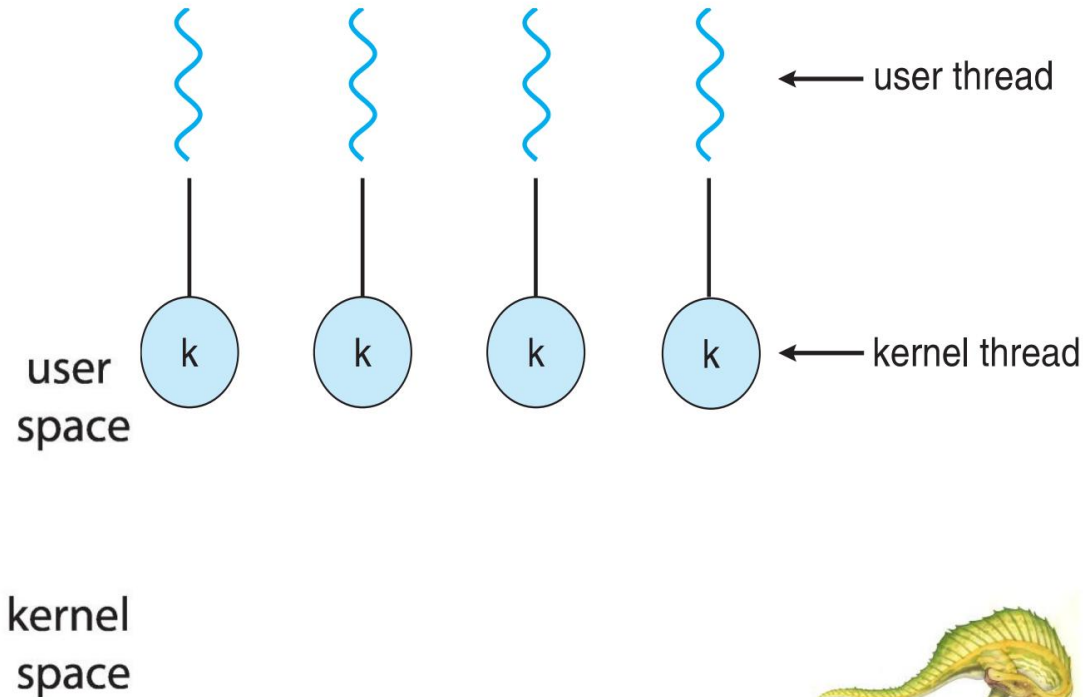
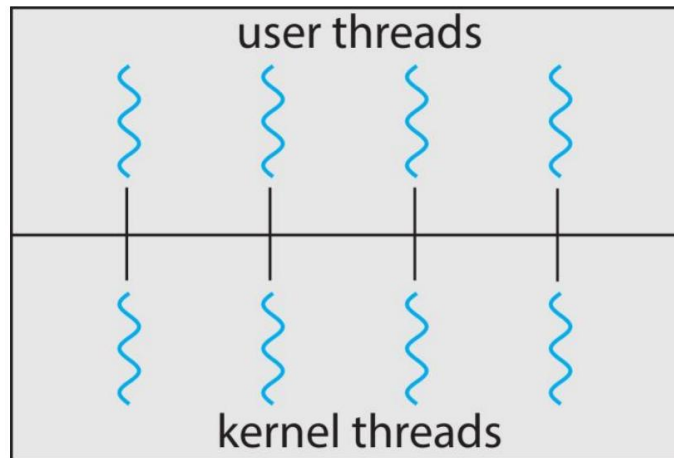
□ قد يكون عدد مؤشرات الترابط لكل عملية محدودًا في بعض الأحيان بسبب النفقات العامة

□ أمثلة

□ ويندوز

□ لينكس

□ Solaris 9 والإصدارات الأحدث





نموذج من كثير إلى كثير

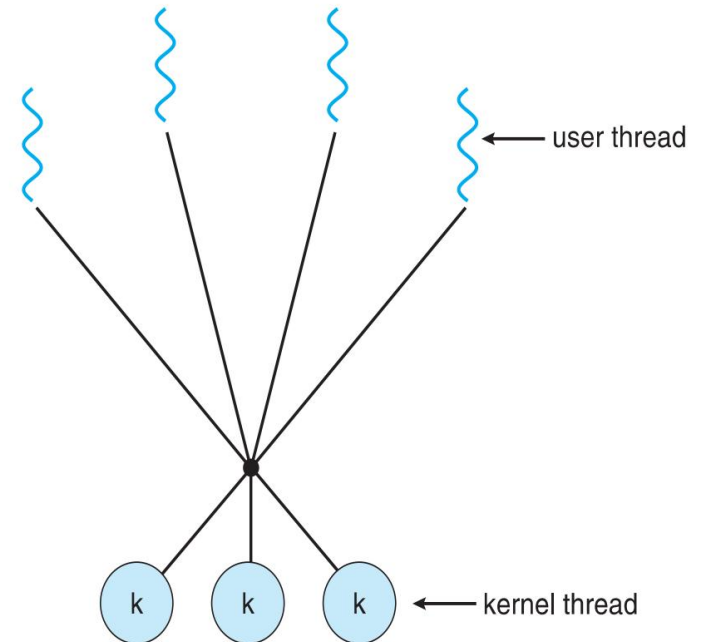
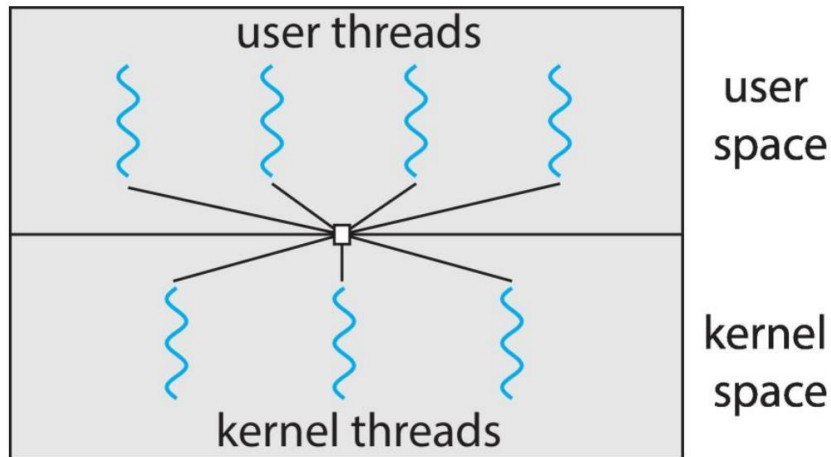
□ يسمح بربط العديد من مؤشرات الترابط على مستوى المستخدم بالعديد من مؤشرات ترابط النواة

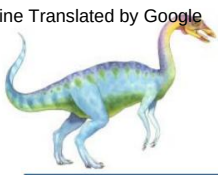
□ يسمح لنظام التشغيل بإنشاء عدد كافٍ من مؤشرات ترابط النواة

Solaris □ قبل الإصدار 9

Windows □ مع الحزمة

خيوط الألياف





نموذج ذو مستويين

□ مشابه لـ $M:M$ إلا أنه يسمح بربط مؤشر ترابط المستخدم بمؤشر ترابط النواة

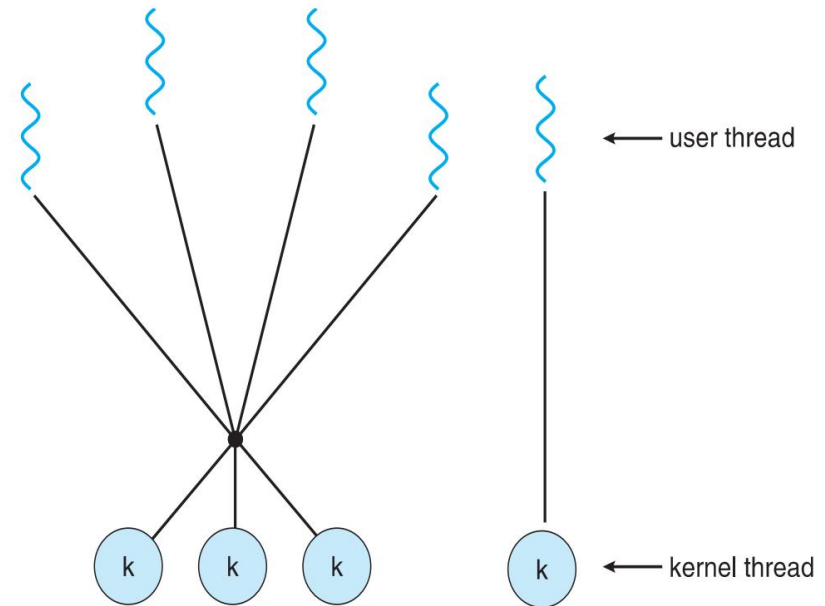
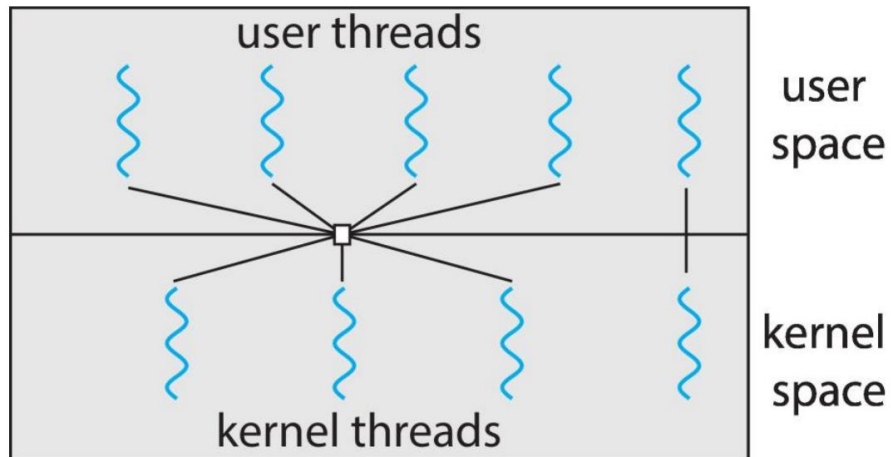
□ أمثلة

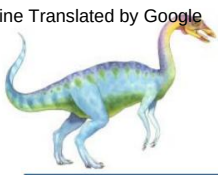
□ إريكس

□ تجربة المستخدم HP

□ Tru64 يونكس

□ Solaris 8 والإصدارات الأقدم





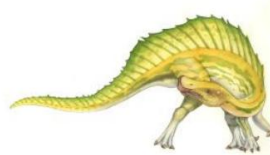
مكتبات الخيوط

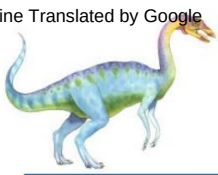
توفر مكتبة الخيوط للمبرمج واجهة برمجة التطبيقات لإنشاء الخيوط وإدارتها

طريقتان أساسيتان للتنفيذ

المكتبة بالكامل في مساحة المستخدم

مكتبة على مستوى النواة يدعمها نظام التشغيل





خيوط P

يمكن توفيرها إما على مستوى المستخدم أو على مستوى النواة

واجهة برمجة تطبيقات بمعيار POSIX (IEEE 1003.1c) لإنشاء الخيوط و

المزامنة

□

لتطبيق

مواصفة

تحدد واجهة برمجة التطبيقات سلوك مكتبة الخيوط، ويتم التنفيذ

حتى تطوير المكتبة

شائع في أنظمة التشغيل (UNIX (Solaris, Linux, Mac OS X





مثال على Pthreads

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```





مثال على Pthreads (تابع)

```
/* get the default attributes */
pthread_attr_init(&attr);
/* create the thread */
pthread_create(&tid,&attr,runner,argv[1]);
/* wait for the thread to exit */
pthread_join(tid,NULL);

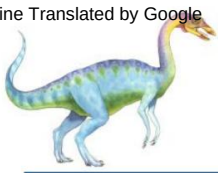
printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```





كود Pthreads لربط 10 خيوط

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```





برنامج C متعدد الخيوط في Windows

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* the thread runs in this separate function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 0; i <= Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;

    if (argc != 2) {
        fprintf(stderr, "An integer parameter is required\n");
        return -1;
    }
    Param = atoi(argv[1]);
    if (Param < 0) {
        fprintf(stderr, "An integer >= 0 is required\n");
        return -1;
    }
}
```





برنامج C متعدد الخيوط في نظام التشغيل Windows (تابع)

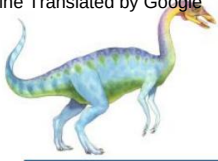
```
/* create the thread */
ThreadHandle = CreateThread(
    NULL, /* default security attributes */
    0, /* default stack size */
    Summation, /* thread function */
    &Param, /* parameter to thread function */
    0, /* default creation flags */
    &ThreadId); /* returns the thread identifier */

if (ThreadHandle != NULL) {
    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n", Sum);
}
}
```





خيوط جافا

□ تتم إدارة مؤشرات ترابط Java بواسطة VM □ ويتم تنفيذها عادةً باستخدام نموذج مؤشرات الترابط الذي توفره

نظام التشغيل الأساسي

□ يمكن إنشاء مؤشرات ترابط Java بواسطة:

```
public interface Runnable
{
    public abstract void run();
}
```

□ توسيع فئة Thread تنفيذ واجهة Runnable





برنامج جافا متعدد الخيوط

```
class Sum
{
    private int sum;

    public int getSum() {
        return sum;
    }

    public void setSum(int sum) {
        this.sum = sum;
    }
}

class Summation implements Runnable
{
    private int upper;
    private Sum sumValue;

    public Summation(int upper, Sum sumValue) {
        this.upper = upper;
        this.sumValue = sumValue;
    }

    public void run() {
        int sum = 0;
        for (int i = 0; i <= upper; i++)
            sum += i;
        sumValue.setSum(sum);
    }
}
```

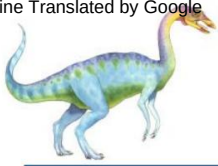




برنامج جافا متعدد الخيوط (تابع)

```
public class Driver
{
    public static void main(String[] args) {
        if (args.length > 0) {
            if (Integer.parseInt(args[0]) < 0)
                System.err.println(args[0] + " must be >= 0.");
            else {
                Sum sumObject = new Sum();
                int upper = Integer.parseInt(args[0]);
                Thread thrd = new Thread(new Summation(upper, sumObject));
                thrd.start();
                try {
                    thrd.join();
                    System.out.println
                        ("The sum of "+upper+" is "+sumObject.getSum());
                } catch (InterruptedException ie) { }
            }
        }
        else
            System.err.println("Usage: Summation <integer value>");
    }
}
```





الترايط الضمني

تزداد شعبيتها مع زيادة عدد الخيوط، وتصبح صحة البرنامج أكثر صعوبة مع الخيوط الصريحة

إنشاء وإدارة الخيوط التي يقوم بها المترجمون و

مكتبات وقت التشغيل بدلاً من المبرمجين

تم استكشاف ثلاث طرق

مجموعات الخيوط

أوبن إم بي

جراند سنترال ديسباتش

تشمل الطرق الأخرى، Microsoft Threading Building Blocks (TBB) وحزمة `java.util.concurrent`





مجموعات الخيوط

❑ إنشاء عدد من المواضيع في مجموعة حيث ينتظرون العمل

❑ المزايا:

❑ عادةً ما يكون أسرع قليلاً في خدمة الطلب الموجود
خيوط ثم إنشاء خيوط جديد

❑ يسمح بعدد الخيوط في التطبيق (التطبيقات)
مرتبط بحجم المسبح

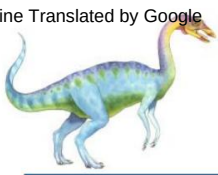
❑ فصل المهمة المراد تنفيذها عن آليات العمل
يتيح إنشاء المهمة استراتيجيات مختلفة لتشغيل المهمة

❑ يمكن جدولة مهام `ieTasks` لتشغيلها بشكل دوري

❑ تدعم واجهة برمجة تطبيقات Windows مجموعات الخيوط:

```
DWORD WINAPI PoolFunction(AVOID Param) {
    /*
     * this function runs as a separate thread.
     */
}
```





أوبن إم بي

مجموعة من توجيهات المترجم و

واجهة برمجة التطبيقات لـ C و FORTRAN و +

يوفر الدعم للتوازي

البرمجة في بيئات الذاكرة المشتركة

تحديد المناطق المتوازية -

كتل من التعليمات البرمجية التي يمكن تشغيلها

بالتوازي

#pragma omp بالتوازي

إنشاء أكبر عدد ممكن من المواضيع

النوى

for(i=0;i<N;i++) { #pragma omp متوازي لـ

ج[i] = أ[i] + ب[i]؛

}

تشغيل حلقة بالتوازي

```
#include <omp.h>
#include <stdio.h>
```

```
int main(int argc, char *argv[])
{
```

```
    /* sequential code */
```

```
    #pragma omp parallel
```

```
    {
        printf("I am a parallel region.");
    }
```

```
    /* sequential code */
```

```
    return 0;
```

```
}
```



محطة غراند سنترال دیسباتش

تقنية Apple لنظامي التشغيل iOS و Mac OS X

ملحقات لغات C و C++ وواجهة برمجة التطبيقات ومكتبة وقت التشغيل

□ يسمح بتحديد الأقسام المتوازية

إدارة معظم تفاصيل الخيوط

الكتلة موجودة في - { } ^ { " ;) أنا كتلة " }

الكتل الموضوعية في قائمة الإرسال

يتم تعيينه إلى مؤشر ترابط متاح في مجموعة مؤشرات الترابط عند إزالته من قائمة الانتظار





محطة غراند سنترال ديسباتش

□ نوعان من طوابير الإرسال:

□ تسلسلي - تتم إزالة الكتل حسب ترتيب FIFO، ويتم وضع قائمة الانتظار لكل عملية، وتسمى **قائمة الانتظار الرئيسية**

□ يمكن للمبرمجين إنشاء طوابير تسلسلية إضافية داخل برنامج

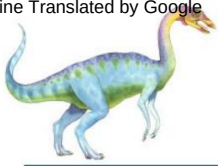
□ متزامن - تمت إزالته بترتيب FIFO ولكن قد يكون هناك العديد تمت إزالتها في وقت واحد

□ ثلاثة طوابير على مستوى النظام بأولويات منخفضة، وافترضية، وعالية

```
dispatch_queue_t queue = dispatch_get_global_queue(
    DISPATCH_QUEUE_PRIORITY_DEFAULT, 0);

dispatch_async(queue, ^{ printf("I am a block."); });
```



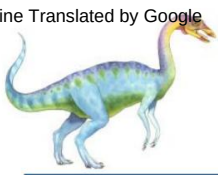


مشاكل الخيوط

❑ دلالات استدعاءات النظام `fork()` و `exec()` معالجة الإشارات ❑ متزامن
❑ إلغاء مؤشر ترابط لمؤشر ترابط مستهدف ❑ غير متزامن أو
مؤجل

❑ تخزين محلي للخيوط
❑ تنشيطات الجدولة





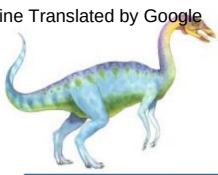
دلالات exec() و fork()

هل تقوم fork() بتكرار الخيط المستدعي فقط أم جميع الخيوط؟
خيوط؟

تحتوي بعض أنظمة UNIX على نسختين من الشوكة

تعمل وظيفة exec() عادةً بشكل طبيعي - استبدال التشغيل
عملية تشمل جميع الخيوط





معالجة الإشارات

تُستخدم الإشارات في أنظمة UNIX لإعلام العملية بوقوع حدث معين.

n يتم استخدام معالج الإشارة لمعالجة الإشارات

1. يتم إنشاء الإشارة بواسطة حدث معين. 2. يتم تسليم الإشارة إلى عملية. 3. تتم

معالجة الإشارة بواسطة أحد معالجي الإشارة:

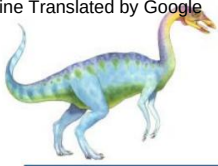
1. الافتراضي

2. مُعرّف من قبل المستخدم

n تحتوي كل إشارة على معالج افتراضي يقوم النواة بتشغيله عند التعامل مع الإشارة. يمكن لمعالج الإشارة المحدد من قبل المستخدم تجاوز المعالج الافتراضي

ل للخيوط المفردة، يتم تسليم الإشارة إلى العملية





معالجة الإشارات (تابع)

أين يجب تسليم الإشارة في حالة تعدد الخيوط؟ ل تسليم الإشارة إلى الخيط الذي ستصل إليه الإشارة

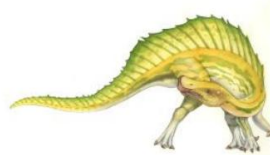
ينطبق

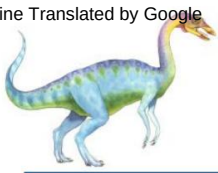
اتوصيل الإشارة إلى كل خيط في العملية

اتوصيل الإشارة إلى خيوط معينة في العملية

لتعيين خيط محدد لاستقبال جميع الإشارات الخاصة بـ

عملية





إلغاء الخيط

□ إنهاء الموضوع قبل الانتهاء منه

□ الموضوع المراد إلغاؤه هو **الموضوع المستهدف**

□ نهجان عامان:

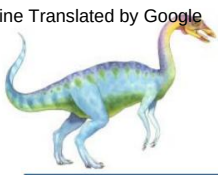
□ يؤدي الإلغاء غير المتزامن إلى إنهاء الخيط المستهدف على الفور

□ يسمح الإلغاء المؤجل للموضوع المستهدف بالتحقق بشكل دوري مما إذا كان يجب إلغاؤه

□ كود Pthread لإنشاء وإلغاء سلسلة:

```
pthread_t tid;  
  
/* create the thread */  
pthread_create(&tid, 0, worker, NULL);  
  
. . .  
  
/* cancel the thread */  
pthread_cancel(tid);
```





إلغاء الخيط (تابع)



يؤدي استدعاء طلبات إلغاء الخيط إلى الإلغاء، ولكن الإلغاء الفعلي يعتمد على حالة الخيط

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous



إذا تم تعطيل إلغاء الموضوع، فإن الإلغاء يظل معلقاً حتى يقوم الموضوع بتمكينه



النوع الافتراضي هو المؤجل



يتم الإلغاء فقط عندما يصل الخيط إلى نقطة الإلغاء

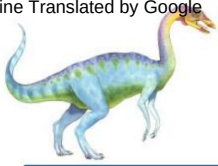
أي pthread_testcancel()

ثم يتم استدعاء معالج التنظيف



في أنظمة Linux، تتم معالجة إلغاء الخيوط من خلال الإشارات





تخزين محلي للخيوط

□ يسمح التخزين المحلي للخيوط (TLS) لكل خيط بالحصول على نسخته الخاصة من البيانات

□ مفيد عندما لا يكون لديك سيطرة على عملية إنشاء الخيوط (على سبيل المثال، عند استخدام مجموعة خيوط)

□ مختلفة عن المتغيرات المحلية

□ المتغيرات المحلية مرئية فقط أثناء استدعاء وظيفة واحدة

□ TLS مرئي عبر استدعاءات الوظيفة

□ مشابهة للبيانات الثابتة

□ TLS فريد لكل خيط





تنشيطات الجدولة

□ يتطلب كل من نموذجي Two-Still و M:M

التواصل للحفاظ على العدد المناسب من مؤشرات ترابط النواة المخصصة للتطبيق

□ عادةً ما تستخدم بنية بيانات وسيطة

بين المستخدم وخيوط النواة -عملية خفيفة الوزن (LWP)

□ يبدو أنه معالج افتراضي يمكن للعملية من خلاله جدول تشغيل مؤشر ترابط المستخدم

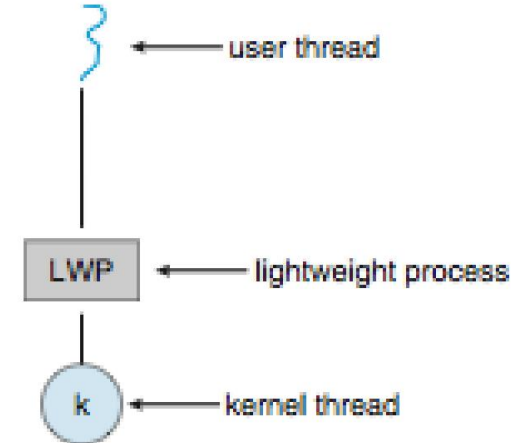
□ كل LWP متصل بخيط النواة

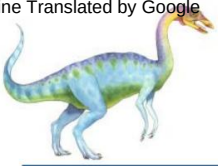
□ كم عدد LWPs التي يجب إنشاؤها؟

□ توفر عمليات تنشيط الجدولة مكالمات إضافية - أ

آلية الاتصال من النواة إلى معالج الاستدعاء في مكتبة الخيوط

□ يسمح هذا الاتصال للتطبيق بالحفاظ على العدد الصحيح لخيوط النواة





أمثلة على أنظمة التشغيل

- ❑ خيوط الويندوز
- ❑ خيوط لينكس





خيوط الويندوز

□ ينفذ Windows واجهة برمجة تطبيقات - Windows واجهة برمجة التطبيقات الأساسية لنظام التشغيل Windows Win 7، Win XP، Win 2000، Win NT، و 98،

□ تنفيذ التعيين الفردي على مستوى النواة

□ يحتوي كل موضوع على

□ معرف الموضوع

□ مجموعة السجلات التي تمثل حالة المعالج

□ فصل مجموعات المستخدم والنواة عند تشغيل الخيط في وضع المستخدم أو وضع النواة

□ منطقة تخزين البيانات الخاصة التي تستخدمها مكتبات وقت التشغيل و

مكتبات الارتباط الديناميكي (DLLs)

□ تُعرف مجموعة السجلات والمكدسات ومنطقة التخزين الخاصة باسم [سياق](#) الخيط





خيوط Windows (تابع)

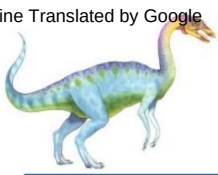
تتضمن هياكل البيانات الأساسية للخيوط ما يلي:

ETHREAD (كتلة الخيط التنفيذي) - تتضمن مؤشرًا إلى العملية التي ينتمي إليها الخيط وإلى **KTHREAD**، مساحة النواة

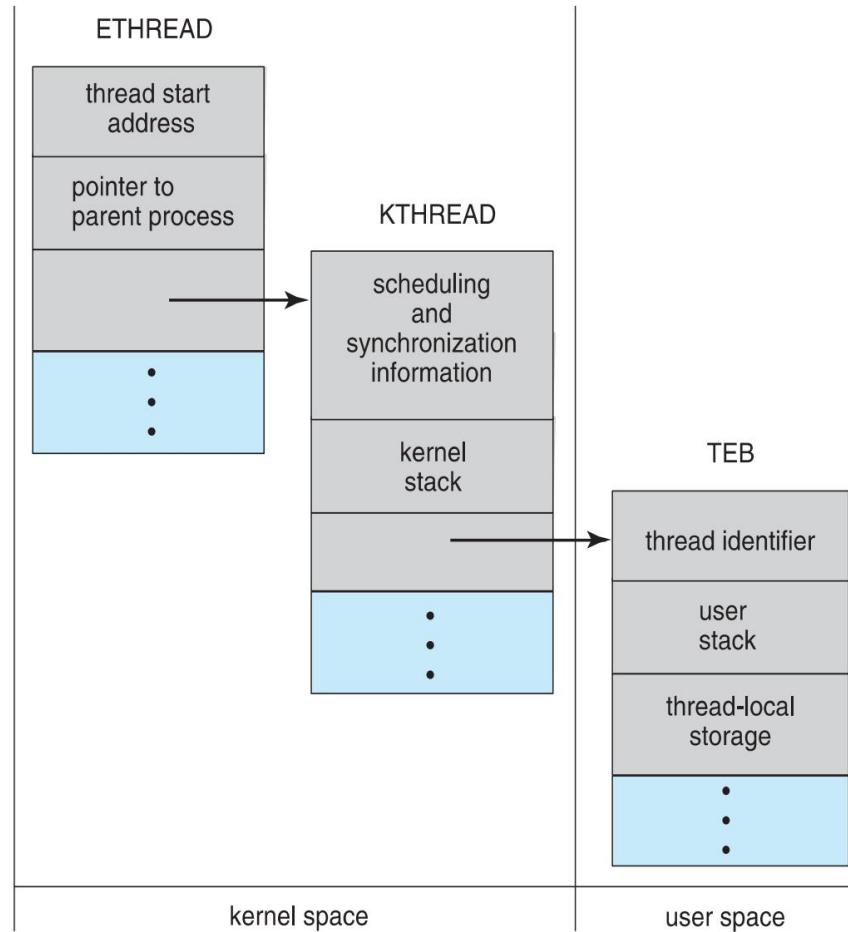
KTHREAD (كتلة خيط النواة) - الجدولة و
معلومات المزامنة، مكس و وضع النواة، مؤشر إلى **TEB**، مساحة النواة

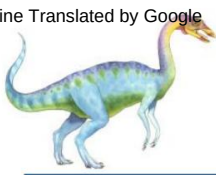
TEB (كتلة بيئة الخيط) - معرف الخيط، ومكس و وضع المستخدم، والتخزين المحلي للخيوط،
في مساحة المستخدم





هياكل بيانات خيوط Windows





خيوط لينكس



يُنشَأ الخيط باسم لينكس



يتم إنشاء الخيط من خلال استدعاء النظام `clone()`



تسمح الدالة `clone()` لمهمة فرعية بمشاركة مساحة عنوان المهمة الأصلية (العملية)



الأعلام تتحكم في السلوك

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.



يشير `struct task_struct` إلى هياكل بيانات العملية (مشاركة أو فريدة)



نهاية الفصل الرابع

